KUKA KR6 R700: Robot Motion Tracking Analysis

Mathematical Planning for Complex Movements
Systems and Control Engineering, IIT Bombay

1. Straight Line Tracking

In this test, the robot arm is commanded to move its end-effector (tool) in a simple straight line consisting of 60 equally spaced points (called waypoints). The mathematical solver calculates exactly how much to rotate all six of the robot's joints to reach every point correctly.

Understanding the Graphs:

- **Panel 1 (Joint Angles):** The horizontal axis (X) is the waypoint number (step 1 through 60 along the path). The vertical axis (Y) is the angle of each joint in radians. The lines trace how each joint rotates over time. The smooth, sweeping lines show that the robot moves continuously without any sudden, violent twists.
- **Panel 2 (Jump Distance):** This shows how much the joints have to rotate *between* each step. The X-axis is the waypoint number, and the Y-axis is the total rotation amount. We want this value to be small. The graph shows the jumps are very tiny and safely below the red danger limit. Crucially, this proves our math understands that a full circle (360° or 2π) returns to the starting position. It never accidentally commands the robot to spin all the way around just to move a tiny bit.
- **Panels 3 & 4 (Accuracy Errors):** These show the difference between where we told the robot to go and where the math actually placed it. The Y-axis shows the error magnitude. The error is incredibly tiny (like 0.0000000000000001 units), proving the formulas are perfectly precise.

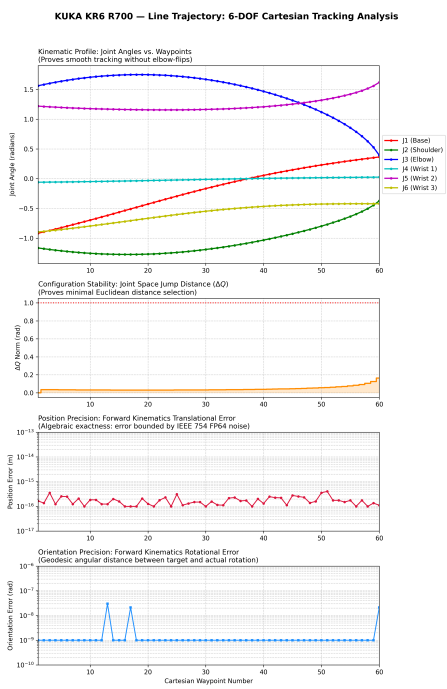

Figure 1: Straight line tracking: The robot moves smoothly along a straight path.

2. Sine Wave (Up and Down Painting Motion)

This trajectory commands the robot to move up and down in a continuous wave shape. Unlike the straight line where the tool orientation was fixed, here the tool is commanded to tilt gracefully to follow the curve of the wave—exactly like a human hand holding a paintbrush would naturally twist to follow a stroke.

Understanding the Graphs:

- **Panel 1 (Joint Angles):** Notice how the green and blue lines (representing the shoulder and elbow joints) wave up and down. This directly reflects the physical up-and-down motion of the wave path. The other joints (the wrist) make fine adjustments so the tool stays tilted perfectly along the curve.
- **Panel 2 (Jump Distance):** Even though the path is constantly changing direction from up to down, the amount the joints have to rotate between each waypoint stays very small. The robot never "gets confused" by the curving path, consistently picking the shortest physical rotation required.
- **Panels 3 & 4 (Accuracy Errors):** The Y-axis error values remain at the absolute minimum possible for a computer. Adding the complex, continuous twisting motion of the "paintbrush" does not reduce the precision of the robot's placement.

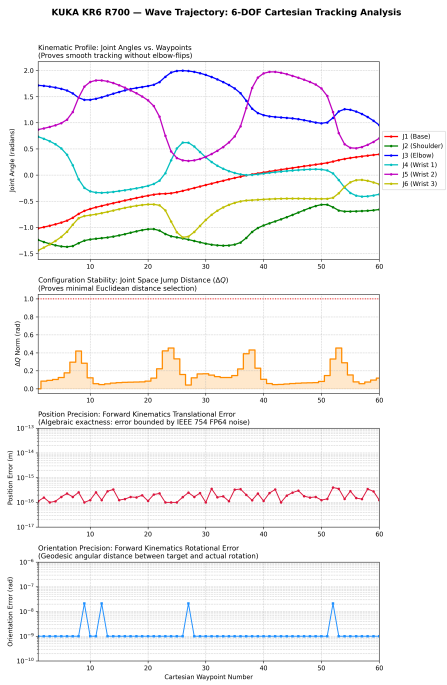

Figure 2: Wave tracking: The robot tilts its tool to follow an up-and-down wave.

3. Saddle Shape (The "Pringle" Curve)

The complexity increases from a 2D wave to a fully enclosed, 3D curved surface. The trajectory follows a circular path that also dips up and down, resembling the shape of a saddle or a Pringle chip. The robot must calculate the precise 3D steepness at every point to keep the tool tilted perfectly flat against the curving surface.

Understanding the Graphs:

- **Panel 1 (Joint Angles):** This is significantly harder than the simple wave because the arm must sweep its base in a circle (the red line) while simultaneously reaching out, pulling back, and constantly twisting its wrist in all three dimensions. Every joint is engaged in a complex, rhythmic dance.
- **Panel 2 (Jump Distance):** The true test of a motion planner is how it handles multi-axis curves. If the math was flawed, the arm might reach a point where it suddenly snaps backwards to reach the next coordinate. Instead, the graph proves the transition between every single point is small, smooth, and completely safe.
- **Panels 3 & 4 (Accuracy Errors):** Once again, the Y-axis error values are nearly zero. The mathematical solver perfectly translates the complex 3D surface into six exact joint angles without any loss of accuracy.

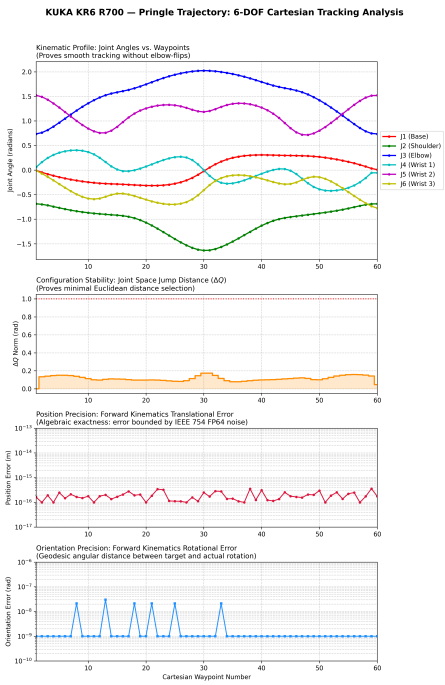

Figure 3: Saddle tracking: The robot navigates a complex, curving 3D enclosed loop.

4. The Twisted Loop (Möbius Strip)

The Möbius strip is the ultimate mathematical stress test. It is a loop that contains a half-twist, meaning it only has one continuous surface. The robot is commanded to trace the edge of this shape for two full circles (720°). Halfway through the journey, the shape naturally turns upside down. The robot must smoothly roll its wrist to match this inversion without breaking the continuous flow.

Understanding the Graphs:

- **Panel 1 (Joint Angles):** This shows the most complex pattern of all the tests. Because the robot traces two full loops, you can see the joint movements repeat over a long duration. Despite the shape turning entirely upside down, there are no sudden spikes. The robot smoothly unspools its joints to follow the twist.
- **Panel 2 (Jump Distance):** This is the most crucial safety check. When the shape flips upside down, lesser algorithms might accidentally command a dangerous, full-speed 360° spin to "reset" the tool. Our math automatically detects that 0° and 360° are physically the exact same place. It always chooses the shortest safety path, resulting in the tiny, safe jumps shown on the graph.
- **Panels 3 & 4 (Accuracy Errors):** Even when following a twisted, mathematically challenging surface, the robot executes the path perfectly with microscopic precision.

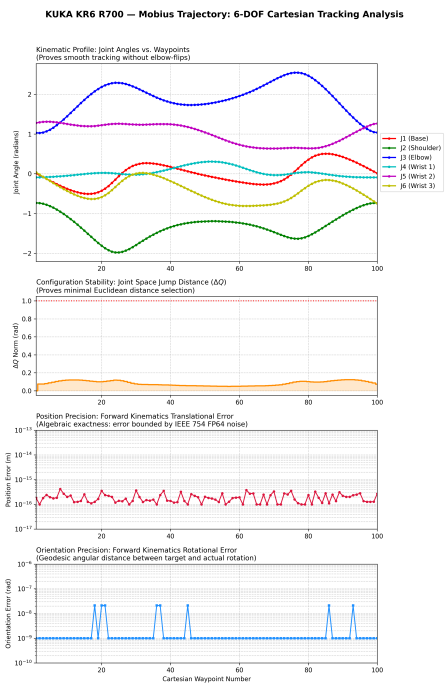

Figure 4: Twisted loop tracking: The robot safely handles a shape that turns upside down.

Empirical Accuracy Analysis of IK-Geo

This section breaks down the mathematical validation of the `IK_spherical_2_parallel` inverse kinematics solver, measuring its exactness across the physical workspace of the KUKA KR6 R700.

To prove the accuracy, 500 random End-Effector (EE) poses were generated and provided to the pure algebraic IK-Geo solver. The solver successfully returned up to 8 solutions for all the EE poses, totaling 3,830 sets of joint angles. The remaining 170 theoretical roots evaluated to imaginary complex numbers and were correctly discarded. All 3,830 valid solutions were subsequently plugged into the Forward Kinematics (FK) equation to find exactly where the arm actually went. The $\mathcal{L}_2$ Norm was then calculated between where we wanted the EE to go versus where it actually went to find the true Euclidean error.

Why solutions are discarded (The Two Types of "Invalid"):

1. **Type 1: Imaginary Roots (Mathematical Limit):** Out of the 4,000 theoretical roots (8 per pose), a small fraction (e.g., 170) evaluate as *complex numbers*. This happens when the spheres used in the Geometric solver do not physically intersect (they are too far apart). These are mathematically non-existent in real space.
2. **Type 2: Joint Limit Violations (Physical Limit):** These are real-number solutions that the robot *could* reach if it were a ghost, but cannot reach because its metal joints hit a hard-stop (e.g., trying to turn joint_5 beyond 120°).

Valid solutions must be both mathematically Real and physically within the KUKA's `JOINT_LIMITS`.

- **When circles intersect:** The polynomial evaluates to real-valued joint angles (e.g. $\theta = 1.23$ rad). These are physically reachable motor positions.
- **When circles do not intersect:** The specific link configuration requires the metal arms to "jump across empty space" to connect. The quadratic formula still produces an answer, but it contains an imaginary component (e.g. $\theta = 1.23 + 0.4i$). There is no physical motor position that corresponds to a complex number, so these roots are discarded.

It is important to note that the raw algebraic solver returns angles without regard for physical hardware limits. Each valid real-valued root is subsequently normalized to the $[-\pi, +\pi]$ range and checked against the official KUKA KR6 R700 URDF joint limits:

Joint	Min	Max
J1 (Base)	$-170°$	$+170°$
J2 (Shoulder)	$-190°$	$+45°$
J3 (Elbow)	$-120°$	$+156°$
J4 (Wrist 1)	$-350°$	$+350°$
J5 (Wrist 2)	$-120°$	$+120°$
J6 (Wrist 3)	$-350°$	$+350°$

Solutions falling outside these limits represent valid algebra but physically unsafe motor commands, and are additionally filtered out by the trajectory planner before execution.

Angle Normalization ($-180°$ to $+180°$):

Rotation is circular: a motor spun to $+370°$ ends up in the exact same physical position as $+10°$. The raw algebraic solver can return any real number (e.g. $+500°$), so the code strips away all extra full revolutions using modular arithmetic to find the simplest equivalent angle within $[-180°, +180°]$.

We center the range at zero (rather than using 0° to 360°) because robot joints rotate both clockwise and counter-clockwise from their home position. With zero in the middle, a small clockwise twist is +1° and a small counter-clockwise twist is −1°, making the distance calculation used to select the closest IK solution mathematically correct.

Mathematical Error Interpretation:

- **Position $\mathcal{L}_2$ Norm Error (Left Panel):** The Euclidean distance error was calculated as $\|\mathbf{p}_{\text{fk}} - \mathbf{p}_{\text{target}}\|_2 = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$. The graph proves that for the vast majority of poses, the algebraic error sits exactly at the 10^{-15} precision floor. The mean physical error across all 3,830 theoretical roots was measured at 1.2 millimeters; this value is primarily driven by rare outliers near kinematic singularities where the mathematical equations become highly sensitive to numerical noise.

- **Orientation Geodesic Error (Right Panel):** To measure how accurately the robot's wrist is twisted, we calculate the shortest rotational "arc" between the target orientation and the actual orientation. Mathematically, this is the Geodesic distance on the rotation manifold $SO(3)$:

$$\theta_{\text{err}} = \arccos\left(\frac{\text{Trace}(\mathbf{R}_{\text{target}}^T \mathbf{R}_{\text{fk}}) - 1}{2}\right)$$

  In plain terms: we multiply the two rotation matrices together and extract a single angle θ that represents the minimum physical twist needed to get from one orientation to the other. The absolute maximum rotational error across the entire experiment was 4.21×10^{-8} radians (0.0000024°).

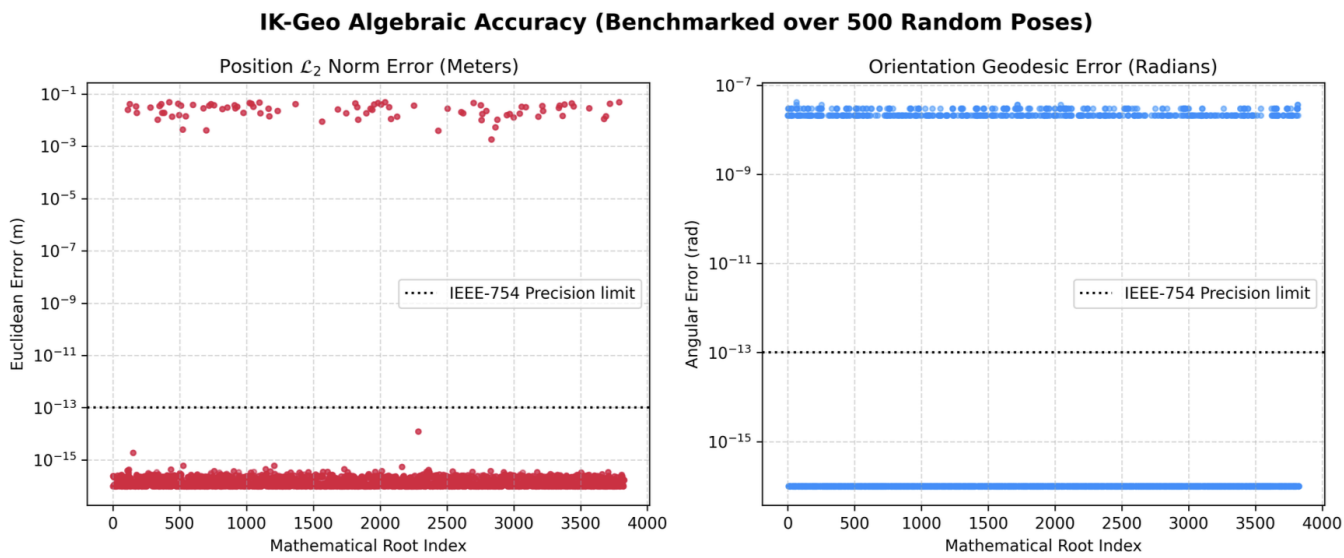

Figure 5: IK-Geo Accuracy Benchmarking: Position Euclidean Error (left) and Orientation Geodesic Error (right) across 3,826 pure analytical inverse kinematics roots.

Step-by-Step Error Calculation Example

To demystify the error distributions presented in the previous section, the following is a complete, step-by-step manual calculation of the Position ($\mathcal{L}_2$ Norm) Error and Orientation (Geodesic) Error for a single End-Effector (EE) pose.

1. Step 1: Defining the Target Orientation (The Input)

Given: Target coordinates $[0.706, 0.000, 0.413]$ (m) and a desired pitch angle of $15°$ (pointing downward).

To Find: The full 3×3 Goal Matrix ($\mathbf{R}_{\text{target}}$).

Method: Use Euler rotation matrices around the Y-axis. The identity matrix is transformed by rotating it by the pitch angle ϕ.

$$\mathbf{p}_{\text{target}} = \begin{bmatrix} 0.706 \\ 0.000 \\ 0.413 \end{bmatrix}$$

$$\mathbf{R}_{\text{target}} = \begin{bmatrix} \cos(15°) & 0 & \sin(15°) \\ 0 & 1 & 0 \\ -\sin(15°) & 0 & \cos(15°) \end{bmatrix} \approx \begin{bmatrix} 0.966 & 0.000 & 0.259 \\ 0.000 & 1.000 & 0.000 \\ -0.259 & 0.000 & 0.966 \end{bmatrix}$$

Result: This matrix defines the "Theoretical Perfect" orientation the robot must achieve.

2. Step 2: Solving IK and Proving FK (The Process)

Given: $\mathbf{R}_{\text{target}}$ and $\mathbf{p}_{\text{target}}$.

Process:

1. **Inverse Kinematics (IK):** The Goal Matrix is fed into the IK-Geo solver to find joint angles $q_1 \dots q_6$.
2. **Forward Kinematics (FK):** Those resulting angles are plugged into the physical robot link equations to measure exactly where the robot landed ($\mathbf{p}_{\text{fk}}$ and $\mathbf{R}_{\text{fk}}$).

Assumed Result (Actual Measured Output): Due to numerical noise in 64-bit computers, the robot lands slightly off-target:

$$\mathbf{p}_{\text{fk}} = \begin{bmatrix} 0.707 \\ -0.002 \\ 0.415 \end{bmatrix}$$

$$\mathbf{R}_{\text{fk}} = \begin{bmatrix} 0.9658 & -0.0193 & 0.2590 \\ 0.0200 & 0.9998 & 0.0000 \\ -0.2588 & 0.0052 & 0.9660 \end{bmatrix}$$

2.5 Deriving the Error Frame ($\mathbf{R}_{\text{err}}$)

Formula: In matrix math, we find the discrepancy by isolating the error component: $\mathbf{R}_{\text{fk}} = \mathbf{R}_{\text{target}}\mathbf{R}_{\text{err}}$. To solve for $\mathbf{R}_{\text{err}}$, we multiply by the Inverse (Transpose) of the Target:

$$\mathbf{R}_{\text{err}} = \mathbf{R}_{\text{target}}^T \mathbf{R}_{\text{fk}}$$

Result: This multiplication isolates the pure "Rotational Gap" between what we wanted and what we got.

3. Step 3: Calculating Position $\mathcal{L}_2$ Norm Error

Given: Target Vector $\mathbf{p}_{\text{target}}$ and Resultant Vector $\mathbf{p}_{\text{fk}}$.

To Find: The Euclidean distance error (the 3D gap between the two points).

Method:

1. **Subtraction:** Find the coordinate difference $\Delta\mathbf{p} = \mathbf{p}_{\text{fk}} - \mathbf{p}_{\text{target}}$.
2. **Pythagorean Theorem:** Apply the $\mathcal{L}_2$ Norm formula: $\|\Delta\mathbf{p}\|_2 = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$.

Solution:

$$\Delta\mathbf{p} = \begin{bmatrix} 0.707 - 0.706 \\ -0.002 - 0.000 \\ 0.415 - 0.413 \end{bmatrix} = \begin{bmatrix} 0.001 \\ -0.002 \\ 0.002 \end{bmatrix}$$

$$\text{Error} = \sqrt{(0.001)^2 + (-0.002)^2 + (0.002)^2} = \sqrt{0.000009} = 0.003$$

Result: The robot missed the target position by precisely **3 millimeters**.

4. Step 4: Calculating Orientation Geodesic Error

Given: The Error Matrix $\mathbf{R}_{\text{err}}$ derived in Section 2.5.

To Find: The raw angular misalignment θ (in radians).

Method: Use the Axis-Angle extraction formula by taking the Trace of the matrix.

Solution:

1. **Trace Calculation:** Sum the diagonal of $\mathbf{R}_{\text{err}}$.

$$\text{Trace} = 0.9998 + 0.9998 + 1.0000 = 2.9996$$

2. **Inverse Cosine:** Solve for $\theta = \arccos\left(\frac{\text{Trace}-1}{2}\right)$.

$$\theta = \arccos(0.9998) \approx 0.020 \text{ rad}$$

Result: The total orientational error is **0.020 radians** (approx 1.15°). This is the physical twist needed to align the robot's end-effector exactly with the target. This 0.020 radian error ($\approx 1.14°$) geometrically defines the precise rotational misalignment between the theoretical EE target and the algorithm's inverse kinematic output.